M05 - Python for Data Engineering

LEARNER BOOK - BEGINNER-FIRST TEACHING RC2

This book teaches before it tests.

Read the explanation, run the demonstration, inspect the result and answer the STOP question before practice.

Contents

DE05-01 - CSV properly: parse, inspect, preserve raw

What you will be able to do

Read a CSV without guessing what commas mean, identify bad/missing values, and preserve the original file.

Why this matters in real Data Engineering

A partner or system may hand you a CSV every day. If your code splits strings on commas or silently replaces blanks with zero, a simple ingestion can corrupt business data before anyone notices.

Picture it this way

Think of CSV as a parcel with packing rules. A comma inside a quoted address belongs inside one field; it is not necessarily a separator. A CSV parser understands the packing rules.

Start from zero

- CSV can contain quoted fields, alternate delimiters, headers, blanks and escaped quote characters.
- **csv.DictReader** reads each row as a dictionary using header names as keys.
- At first, values arrive as text. The string **''** is not automatically numeric zero.
- A missing value, zero value and missing column are different states.
- Treat incoming data as **raw evidence**. Read it; do not overwrite it while discovering defects.
- First-pass checks: row count, columns, required blanks, parseability and repeated business keys.

Teacher demonstration

```
import csv
from pathlib import Path
path = Path("DATA/orders.csv")
with path.open("r", encoding="utf-8", newline="") as f:
    rows = list(csv.DictReader(f))
print("rows:", len(rows))
print("first row:", rows[0])
print("blank amount rows:", sum(r["amount"] == "" for r in rows))
```

Read it like English

- Path represents the file location.
- newline='' lets the csv module control newline parsing.
- DictReader uses order_id/customer_id/amount/order_date as dictionary keys.
- The training file has 6 raw rows, one blank amount, one text value 'invalid', and O02 twice.

Expected check

check	expected
raw rows	6
blank amount rows	1
non-numeric amount	O04 -> invalid
repeated order_id	O02 x2

Common beginner traps

- Using line.split(',') for general CSV.
- Turning blank amount into 0 without a business rule.
- Writing cleaned rows back into orders.csv.

Now you do a similar task

Read DATA/orders.csv. Print header names, raw row count, blank amounts and repeated order IDs. Do not clean/delete anything.

How to prove it

- Save actual code/command.
- Save one independent validation.
- State important input/output grain or item meaning.
- Debug disagreements before moving on.

STOP - check your understanding

Why is an empty amount not the same as amount = 0?

Do not turn the page until you can answer in your own words.

DE05-02 - JSON and JSONL from zero

What you will be able to do

Recognize JSON objects/lists and process JSON Lines one record at a time.

Why this matters in real Data Engineering

APIs and event systems commonly return JSON. If you cannot distinguish an object from a list, correct JSON still feels mysterious.

Picture it this way

JSON is nested labeled boxes. Curly braces are one object with named fields. Square brackets are an ordered list. JSONL is a notebook where each line is one complete JSON object.

Start from zero

- A JSON **object** becomes a Python dict; an **array** becomes a list.
- JSON null becomes Python None.
- **json.load** reads from a file; **json.loads** parses JSON text already in memory.
- JSONL/NDJSON stores one complete JSON object per line.
- Before indexing nested data, inspect type, keys, length and a sample.
- Required vs optional keys are contract decisions; do not just crash on every absent optional key.

Teacher demonstration

```
import json
from pathlib import Path
customers = json.loads(
    Path("DATA/customers.json").read_text(encoding="utf-8")
)
print(type(customers))
print(customers.keys())
print(customers["customers"][0])
for line in Path("DATA/events.jsonl").read_text(encoding="utf-8").splitlines():
    event = json.loads(line)
    print(event["order_id"], event["status"])
```

Read it like English

- Top-level customers is a dict.
- customers['customers'] is a list.
- Each element is one customer dict.
- events.jsonl is parsed one line at a time.
- The event file contains two events for O01 and one for O02.

Expected check

check	expected
top-level type	dict
customer rows	5
event rows	3
events for O01	2

Common beginner traps

- Writing customers[0] when top level is a dict.
- Using eval() on JSON.
- Assuming nested optional keys always exist.

Now you do a similar task

Build customer_id -> region from customers.json. Then count JSONL events per order_id.

How to prove it

- Save actual code/command.
- Save one independent validation.
- State important input/output grain or item meaning.
- Debug disagreements before moving on.

STOP - check your understanding

If JSON starts with {, what Python structure do you normally expect? What if it starts with [?

Do not turn the page until you can answer in your own words.

DE05-03 - Paths, folders and deterministic batch files

What you will be able to do

Discover only intended files, process them predictably, and archive/quarantine only after the correct decision.

Why this matters in real Data Engineering

Batch pipelines read landing folders. Backup/temp files or unpredictable ordering can make the same run behave differently.

Picture it this way

A landing folder is an arrivals area. Select only parcels matching the manifest; after processing, successful and rejected items go to different destinations.

Start from zero

- **Path** makes file/folder handling easier to read.
- **glob('items_*.csv')** discovers matching files.
- Use **sorted(...)** when order matters.
- Filename patterns are part of the input contract.
- Useful states: landing -> processing -> archive for success; quarantine for invalid input.
- Do not archive the only input copy before successful output/validation.

Teacher demonstration

```
from pathlib import Path
import csv
folder = Path("DATA/batch")
files = sorted(folder.glob("items_*.csv"))
for path in files:
    with path.open("r", encoding="utf-8", newline="") as f:
        rows = list(csv.DictReader(f))
    print(path.name, len(rows))
```

Read it like English

- Three training files match the pattern.
- Sorted names give 20260820, 20260821, 20260822 order.
- Expected row counts are 2, 2, 1.
- Each count remains attached to its filename.

Expected check

file	rows
items_20260820.csv	2
items_20260821.csv	2
items_20260822.csv	1

Common beginner traps

- Processing every *.csv including temp/backup files.
- Depending on OS folder order.
- Archiving before validation succeeds.

Now you do a similar task

Print a sorted manifest filename + row_count and total business rows. Explain one reason to quarantine instead of archive.

How to prove it

- Save actual code/command.
- Save one independent validation.
- State important input/output grain or item meaning.
- Debug disagreements before moving on.

STOP - check your understanding

Why is sorted(glob(...)) useful for reproducibility?

Do not turn the page until you can answer in your own words.

DE05-04 - pandas profiling before transformation

What you will be able to do

Inspect a DataFrame before cleaning so you know what you are changing.

Why this matters in real Data Engineering

pandas can transform data very quickly. A wrong inferred type or duplicate key can be amplified before you notice.

Picture it this way

Before repairing a machine, count and inspect the parts. DataFrame profiling is that inspection.

Start from zero

- **pd.read_csv** loads CSV into a DataFrame.
- **shape** gives rows/columns; **columns** gives names; **dtypes** gives inferred types.
- **head()** samples rows; **isna().sum()** counts missing values.
- **duplicated('order_id')** helps inspect repeated keys.
- Inferred dtype is evidence, not business truth.
- Keep raw_df separate from clean_df when raw evidence matters.

Teacher demonstration

```
import pandas as pd
raw_df = pd.read_csv("DATA/orders.csv")
print("shape:", raw_df.shape)
print("columns:", raw_df.columns.tolist())
print(raw_df.dtypes)
print(raw_df.isna().sum())
print("duplicate later rows:", raw_df.duplicated("order_id").sum())
```

Read it like English

- Training shape is 6 rows x 4 columns.
- One amount becomes missing in pandas.
- The text 'invalid' prevents a clean numeric amount dtype.
- duplicated().sum() reports one later duplicate; keep=False shows all duplicate participants.

Expected check

check	expected
shape	(6, 4)
amount missing	1
duplicated later occurrence	1
repeated key	O02

Common beginner traps

- Calling dropna/drop_duplicates immediately.
- Assuming amount is valid because some values look numeric.
- Using pandas without stating row grain.

Now you do a similar task

Save a profile: shape, columns, dtypes, null counts, duplicate-key rows and a sample. No cleaning yet.

How to prove it

- Save actual code/command.
- Save one independent validation.
- State important input/output grain or item meaning.
- Debug disagreements before moving on.

STOP - check your understanding

Why does an inferred dtype not prove a column is business-valid?

Do not turn the page until you can answer in your own words.

DE05-05 - Cleaning with pandas and rejected rows

What you will be able to do

Convert values deliberately and split valid/rejected records with explicit reasons.

Why this matters in real Data Engineering

Silently dropping bad rows produces clean-looking output with invisible data loss.

Picture it this way

A rejected bag is routed aside with a reason; it does not disappear. Pipelines need the same behavior.

Start from zero

- **pd.to_numeric(..., errors='coerce')** turns conversion failures into missing values.
- That NaN is evidence of a bad/missing source value, not an automatic fix.
- **duplicated(key, keep=False)** marks all rows in duplicate-key problems.
- Keep a **reject_reason** so another engineer knows why the row was rejected.
- A record may fail multiple rules; choose whether to store one primary reason or all reasons.
- Do not drop duplicate keys before the business defines a winner rule.

Teacher demonstration

```
import pandas as pd
df = pd.read_csv("DATA/orders.csv", dtype=str)
df["amount_num"] = pd.to_numeric(df["amount"], errors="coerce")
df["is_duplicate_id"] = df.duplicated("order_id", keep=False)
def reason(row):
    reasons = []
    if pd.isna(row["amount_num"]):
        reasons.append("invalid_or_missing_amount")
    if row["is_duplicate_id"]:
        reasons.append("duplicate_order_id")
    return "|".join(reasons)
df["reject_reason"] = df.apply(reason, axis=1)
valid = df[df["reject_reason"] == ""].copy()
rejected = df[df["reject_reason"] != ""].copy()
print("valid:", len(valid))
print("rejected:", len(rejected))
```

Read it like English

- Both O02 rows are duplicate-key rejects.
- O03 is missing amount.
- O04 has non-numeric amount.
- Only O01 and O05 pass these strict training rules.
- Expected valid=2, rejected=4.

Expected check

result	expected
valid rows	2
rejected rows	4
O02	duplicate_order_id

result	expected
O03/O04	invalid_or_missing_amount

Common beginner traps

- drop_duplicates() before inspection.
- fillna(0) on amount.
- Only 'invalid=True' with no usable reason.

Now you do a similar task

Create valid_orders and rejected_orders with reasons. Reconcile valid + rejected = raw rows and keep original amount text.

How to prove it

- Save actual code/command.
- Save one independent validation.
- State important input/output grain or item meaning.
- Debug disagreements before moving on.

STOP - check your understanding

What does errors='coerce' do, and why does it not make the source value valid?

Do not turn the page until you can answer in your own words.

DE05-06 - pandas merge, cardinality and aggregation

What you will be able to do

Enrich data with a customer master while guarding merge cardinality and preserving unmatched evidence.

Why this matters in real Data Engineering

SQL join-grain reasoning applies directly to pandas merges.

Picture it this way

You attach one customer card to each order. If the customer master has two cards for one ID, an order can multiply. The merge should expose that.

Start from zero

- **how='left'** preserves left rows.
- **validate='many_to_one'** says many orders may map to at most one customer row.
- **indicator=True** adds _merge to expose matched/unmatched states.
- Compare row count before/after enrichment.
- Unmatched IDs should be visible, not silently discarded.
- Group only after deciding which population is valid.

Teacher demonstration

```
import json
from pathlib import Path
import pandas as pd
orders = pd.read_csv("DATA/orders.csv", dtype=str)
customers = pd.DataFrame(
    json.loads(Path("DATA/customers.json").read_text(encoding="utf-8"))["customers"]
)
merged = orders.merge(
    customers, on="customer_id", how="left",
    validate="many_to_one", indicator=True
)
print("before:", len(orders))
print("after:", len(merged))
print("unmatched:", (merged["_merge"] != "both").sum())
```

Read it like English

- Customer master has one row per C01-C05.
- All six training order rows reference C01-C05.
- Expected 6 -> 6 and unmatched=0.
- Duplicate right-side customer keys would raise instead of silently multiplying.

Expected check

check	expected
rows before	6
rows after	6
unmatched	0

Common beginner traps

- Merging on readable names instead of keys.
- Ignoring before/after row counts.
- Grouping invalid rows before defining population.

Now you do a similar task

Merge valid orders to customers with many_to_one + indicator. Summarize valid amount by region and separately report unmatched IDs.

How to prove it

- Save actual code/command.
- Save one independent validation.
- State important input/output grain or item meaning.
- Debug disagreements before moving on.

STOP - check your understanding

What specific mistake can validate='many_to_one' detect?

Do not turn the page until you can answer in your own words.

DE05-07 - Data validation: record, batch and relationship rules

What you will be able to do

Turn business expectations into reusable checks and distinguish row-level rejects from unsafe whole-batch failures.

Why this matters in real Data Engineering

Validation is the boundary between 'data arrived' and 'data is acceptable downstream'.

Picture it this way

A damaged box can be quarantined; a missing shipment manifest may make the entire truck unsafe.

Start from zero

- A **record rule** checks one record: amount >= 0.
- A **batch rule** checks the dataset: required columns exist, key uniqueness.
- A **relationship rule** checks references such as customer_id in master.
- Missing critical schema usually stops the batch.
- Report rule name, affected count, sample keys and whether processing may continue.
- Same input + same rules should produce the same validation result.

Teacher demonstration

```
def require_columns(df, required):
    missing = set(required) - set(df.columns)
    if missing:
        raise ValueError(f"Missing required columns: {sorted(missing)}")
def duplicate_keys(df, key):
    return df.loc[df.duplicated(key, keep=False), key].tolist()
require_columns(df, ["order_id","customer_id","amount","order_date"])
print(duplicate_keys(df, "order_id"))
```

Read it like English

- Missing schema stops because downstream interpretation is unsafe.
- duplicate_keys returns evidence instead of deleting rows.
- O02 should appear as duplicate evidence.

Expected check

rule	training result
required columns	PASS
duplicate key	O02
customer relationship C01-C05	PASS

Common beginner traps

- One generic invalid flag.
- Continuing after a critical column is missing.
- Removing invalid rows without counts/reasons.

Now you do a similar task

Write require_columns, duplicate_key_report and unknown_customer_report. Test each with good and deliberately bad mini-DataFrames.

How to prove it

- Save actual code/command.
- Save one independent validation.
- State important input/output grain or item meaning.
- Debug disagreements before moving on.

STOP - check your understanding

Give one row-level failure and one batch-level failure.

Do not turn the page until you can answer in your own words.

DE05-08 - HTTP and REST APIs from zero

What you will be able to do

Understand client/server, endpoint, parameters, headers, status and body before coding requests.

Why this matters in real Data Engineering

Without this model, requests.get looks like magic and pagination/auth/status codes feel random.

Picture it this way

Your script is a customer at a service counter. The endpoint is the counter, method is the requested action, parameters are form details, response is receipt + result.

Start from zero

- Python is the HTTP **client**; API program is the **server**.
- A URL can have scheme, host, port, path and query parameters.
- Endpoint example: GET /orders.
- GET usually retrieves; other methods exist but follow the API contract.
- A response has status code, headers and body.
- HTTP 200 does not prove returned business data is clean.

Teacher demonstration

```
# Terminal A:
# python DATA/M05_LOCAL_API_SERVER.py
#
# Then inspect in browser:
# http://127.0.0.1:8765/orders?page=1
```

Read it like English

- 127.0.0.1 is your own PC.
- 8765 is training port.
- /orders is path/endpoint.
- page=1 is a query parameter.
- Response has items and next_page.

Expected check

concept	example
method	GET
endpoint	/orders
query parameter	page=1
status	200
top-level keys	items, next_page

Common beginner traps

- Treating page number as endpoint.
- Assuming first response has all records.
- Treating 200 as data-quality proof.

Now you do a similar task

Start local API. For /orders?page=1 identify client, server, host, port, endpoint, query parameter, status, item count and next-page signal.

How to prove it

- Save actual code/command.
- Save one independent validation.
- State important input/output grain or item meaning.
- Debug disagreements before moving on.

STOP - check your understanding

What is the difference between /orders and ?page=1?

Do not turn the page until you can answer in your own words.

DE05-09 - Calling APIs safely with requests

What you will be able to do

Call an endpoint with requests using parameters, timeouts and explicit HTTP failure handling.

Why this matters in real Data Engineering

A script that only works when everything is perfect is not reliable ingestion.

Picture it this way

Timeout says how long you will wait. raise_for_status checks the receipt for HTTP failure before treating the body as normal.

Start from zero

- **requests.get** performs GET.
- Use params instead of manually concatenating query strings.
- Always choose a timeout.
- **raise_for_status()** turns many 4xx/5xx statuses into exceptions.
- Call .json() only after success when JSON is expected.
- Put repeated request behavior in a small function.

Teacher demonstration

```
import requests
BASE_URL = "http://127.0.0.1:8765"
def fetch_page(page):
    r = requests.get(
        f"{BASE_URL}/orders",
        params={"page": page},
        timeout=5
    )
    r.raise_for_status()
    return r.json()
page1 = fetch_page(1)
print(len(page1["items"]))
print(page1["next_page"])
```

Read it like English

- Training endpoint returns 3 items per page.
- Page 1 has 3 items and next_page=2.
- If server is not running, connection error is evidence to diagnose.

Expected check

check	expected
page 1 items	3
next_page	2

Common beginner traps

- No timeout.
- Calling .json() before handling HTTP failure.
- Hardcoding URL/timeout repeatedly.

Now you do a similar task

Write fetch_page(page, timeout=5). Print status and item count for pages 1 and 2.

How to prove it

- Save actual code/command.
- Save one independent validation.
- State important input/output grain or item meaning.
- Debug disagreements before moving on.

STOP - check your understanding

Why should ingestion requests have a timeout?

Do not turn the page until you can answer in your own words.

DE05-10 - Pagination and authentication without leaking secrets

What you will be able to do

Fetch every page and use credentials from environment variables rather than committed source.

Why this matters in real Data Engineering

Fetching page 1 only silently truncates data; hardcoded credentials create security risk.

Picture it this way

Pagination is a multi-page form. An API key is a badge: use it, but do not publish or print the badge number.

Start from zero

- Pagination may use page numbers, offsets, cursors or next URLs.
- Follow the API-provided next signal.
- Keep page evidence so incomplete runs are visible.
- Authentication identifies the caller; authorization controls access.
- Credentials belong outside committed code.
- Training key is fake, but habit matches real secret handling.

Teacher demonstration

```
import os
import requests
page = 1
all_items = []
while page is not None:
    data = fetch_page(page)
    all_items.extend(data["items"])
    page = data["next_page"]
print("total:", len(all_items))
api_key = os.environ["TRAINING_API_KEY"]
r = requests.get(
    f"{BASE_URL}/secure-products",
    headers={"X-API-Key": api_key},
    timeout=5
)
r.raise_for_status()
print(len(r.json()["items"]))
```

Read it like English

- Training /orders contains 7 items across 3 pages.
- Loop ends when next_page is None.
- Set TRAINING_API_KEY=training-key-2026 in shell.
- Correct training key returns 2 products; wrong/missing key returns 401.

Expected check

check	expected
all /orders items	7
secure products	2
wrong/missing key	401

Common beginner traps

- Stopping after first page.
- Infinite loop by not updating page.
- Putting real key in source/log.

Now you do a similar task

Fetch all pages and assert total=7. Use TRAINING_API_KEY from shell to call secure-products. Do not print the key.

How to prove it

- Save actual code/command.
- Save one independent validation.
- State important input/output grain or item meaning.
- Debug disagreements before moving on.

STOP - check your understanding

Why is page 1 returning 200 not proof the extraction is complete?

Do not turn the page until you can answer in your own words.

DE05-11 - Retries, rate limits and partial failure

What you will be able to do

Retry only transient-looking failures, respect rate limits, and mark partial extractions incomplete.

Why this matters in real Data Engineering

Unbounded retries can hide permanent errors and create duplicate/partial work.

Picture it this way

If a shop says 'temporarily unavailable', retry later. If your ID is invalid, presenting it 100 times does not help.

Start from zero

- Retry candidates can include selected connection errors, 429 and some 5xx.
- 401 invalid credentials usually needs repair, not repeated identical requests.
- Bound attempts.
- For 429, respect Retry-After when provided.
- Backoff waits between attempts.
- If one required page fails, the run is incomplete even if earlier pages succeeded.

Teacher demonstration

```
import time
import requests
def get_with_retry(url, max_attempts=3):
    for attempt in range(1, max_attempts + 1):
        r = requests.get(url, timeout=2)
        print("attempt", attempt, "status", r.status_code)
        if r.status_code == 429:
            time.sleep(float(r.headers.get("Retry-After", "1")))
            continue
        if 500 <= r.status_code < 600:
            if attempt == max_attempts:
                r.raise_for_status()
            time.sleep(0.2 * attempt)
            continue
        r.raise_for_status()
        return r.json()
    raise RuntimeError("No successful response")
```

Read it like English

- /flaky-orders returns 500 once then succeeds.
- /rate-limited returns 429 once with Retry-After then succeeds.
- Attempts are bounded.
- Real code should also define retryable requests exceptions and run/page logging.

Expected check

endpoint	behavior
/flaky-orders	500 then success
/rate-limited	429 then success
attempts	bounded

Common beginner traps

- Retrying 401 forever.
- while True with no policy.
- Claiming success after partial pagination.
- Logging huge sensitive bodies.

Now you do a similar task

Exercise flaky/rate endpoints. Then write your retry/non-retry policy and incomplete-run rule.

How to prove it

- Save actual code/command.
- Save one independent validation.
- State important input/output grain or item meaning.
- Debug disagreements before moving on.

STOP - check your understanding

Why should a 401 usually be repaired instead of retried identically?

Do not turn the page until you can answer in your own words.

DE05-12 - Python + PostgreSQL with SQLAlchemy

What you will be able to do

Load validated rows into PostgreSQL using environment config, parameterized SQL and transactions.

Why this matters in real Data Engineering

Database loads must avoid embedded credentials, unsafe SQL string building and duplicate rerun behavior.

Picture it this way

Parameterized SQL is a safe intake form: values go into labeled blanks instead of being pasted into SQL command text.

Start from zero

- Reuse PostgreSQL from earlier; do not reinstall blindly.
- SQLAlchemy provides engine/connection abstractions; Psycopg is a modern PostgreSQL driver.
- Store full **DATABASE_URL** outside source.
- Use SQLAlchemy **text** with named parameters.
- **engine.begin()** commits on success and rolls back if an exception escapes.
- Database constraints remain a second line of defense.
- Rerunnable loads need a key/conflict policy so keys do not double.

Teacher demonstration

```
import os
from sqlalchemy import create_engine, text
engine = create_engine(os.environ["DATABASE_URL"])
create_sql = text(
    "CREATE TABLE IF NOT EXISTS m05_orders "
    "(order_id TEXT PRIMARY KEY, "
    "amount NUMERIC NOT NULL CHECK (amount >= 0))"
)
upsert_sql = text(
    "INSERT INTO m05_orders(order_id, amount) "
    "VALUES (:order_id, :amount) "
    "ON CONFLICT (order_id) DO UPDATE "
    "SET amount = EXCLUDED.amount"
)
with engine.begin() as conn:
    conn.execute(create_sql)
    conn.execute(upsert_sql, {"order_id": "LAB1", "amount": 100})
```

Read it like English

- Connection URL is read at runtime.
- Named parameters keep data separate from SQL syntax.
- Primary key identifies business row.
- ON CONFLICT makes same-key rerun controlled.
- Transaction groups the work.

Expected check

check	expected
parameters	:order_id, :amount

check	expected
same LAB1 twice	one LAB1 key
negative amount	DB CHECK rejects

Common beginner traps

- Hardcoding database password.
- f-stringing data into INSERT SQL.
- Loading invalid rows first.
- Assuming transaction alone deduplicates.

Now you do a similar task

Create m05_test_orders with PK + non-negative CHECK. Insert two validated rows with parameters. Rerun and prove keys do not double.

How to prove it

- Save actual code/command.
- Save one independent validation.
- State important input/output grain or item meaning.
- Debug disagreements before moving on.

STOP - check your understanding

What different problems do parameterized SQL and a PRIMARY KEY solve?

Do not turn the page until you can answer in your own words.

DE05-13 - Configuration, environment variables and secret hygiene

What you will be able to do

Separate code from environment-specific config and keep secrets out of Git/logs.

Why this matters in real Data Engineering

The same program should run in different environments without source-code edits for every URL/path/credential.

Picture it this way

Code is the recipe; config is the kitchen setting; a secret is the locked pantry key.

Start from zero

- Config: API URL, input path, timeout, log level, database URL.
- Secret: password, token, private key.
- **os.getenv** can use a safe default; **os.environ[name]** can require a value.
- .env may help local development but real secrets must be gitignored.
- .env.example contains names/placeholders only.
- Do not print whole config objects if they may include secrets.

Teacher demonstration

```
import os
API_BASE_URL = os.getenv(
    "API_BASE_URL",
    "http://127.0.0.1:8765"
)
DATABASE_URL = os.environ["DATABASE_URL"]
print("API base:", API_BASE_URL)
print("Database URL configured:", bool(DATABASE_URL))
# Do not print DATABASE_URL itself.
```

Read it like English

- API base has a safe local default.
- DATABASE_URL is required so wrong-database fallback is avoided.
- Code confirms presence without exposing credential content.

Expected check

artifact	safe behavior
.env.example	placeholders only
.env	gitignored
logs	no token/password values

Common beginner traps

- Committing a real .env.
- Printing DATABASE_URL/token.
- Hardcoding machine-specific paths.

Now you do a similar task

Create .env.example + .gitignore; set API_BASE_URL in shell and read it. Do not put real credentials in example.

How to prove it

- Save actual code/command.
- Save one independent validation.
- State important input/output grain or item meaning.
- Debug disagreements before moving on.

STOP - check your understanding

What is the difference between configuration and a secret?

Do not turn the page until you can answer in your own words.

DE05-14 - Logging, testing and reusable pipeline code

What you will be able to do

Use useful logs, small functions and targeted tests instead of print-only monoliths.

Why this matters in real Data Engineering

When a pipeline fails later, you need context and reproducible behavior checks.

Picture it this way

Logs are the flight recorder; tests are pre-flight checks for small components.

Start from zero

- Use logging levels like INFO/WARNING/ERROR.
- Useful context: run ID, source/page, input/rejected/loaded counts, status/error reason.
- Never log secrets or huge sensitive data.
- Pure functions are easy to test.
- pytest discovers test functions and reports pass/fail.
- Separate extract/validate/transform/load enough to reason about each.

Teacher demonstration

```
import logging
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)s %(message)s"
)
def is_valid_amount(value):
    return value >= 0
def test_is_valid_amount():
    assert is_valid_amount(0)
    assert is_valid_amount(100)
    assert not is_valid_amount(-1)
logging.info("validation rule ready")
```

Read it like English

- Function has no network/database side effect.
- Tests cover boundary, normal and invalid values.
- Log records a useful event without a credential.

Expected check

check	expected
is_valid_amount(0)	True
is_valid_amount(-1)	False
pytest	PASS

Common beginner traps

- Only print statements.
- Tests with no boundaries.
- One giant side-effect-heavy function.

Now you do a similar task

Write 3 pytest cases for one validation rule and INFO logs around a tiny extract function. Confirm no secret appears.

How to prove it

- Save actual code/command.
- Save one independent validation.
- State important input/output grain or item meaning.
- Debug disagreements before moving on.

STOP - check your understanding

Why are small pure validation functions easier to test?

Do not turn the page until you can answer in your own words.

DE05-15 - Integrated Python data-engineering pipeline

What you will be able to do

Assemble a rerunnable mini-pipeline: extract -> raw -> validate -> transform -> load -> verify -> log.

Why this matters in real Data Engineering

This is the bridge from Python syntax to Data Engineering.

Picture it this way

A reliable pipeline is a factory with checkpoints: receive, preserve, reject/accept, transform, load and reconcile.

Start from zero

- **Extract**: fetch every page/file and know whether extraction completed.
- **Preserve raw**: save incoming evidence before business transformation when practical.
- **Validate**: schema/types/ranges/uniqueness/relationships.
- **Transform**: create clean business shape after validity decisions.
- **Load**: transaction + constraints + key/conflict policy.
- **Verify**: reconcile extracted, valid, rejected and loaded counts.
- **Log**: run start/end/counts/status with no secrets.
- Same input rerun must not create unintended duplicate target keys.

Teacher demonstration

```
# High-level structure only - not the independent solution
def extract_all_orders(...):
    ...
def save_raw_pages(...):
    ...
def validate_orders(...):
    ...
def transform_orders(...):
    ...
def load_orders(...):
    ...
def main():
    # extract -> raw -> validate -> transform
    # -> load -> verify -> log
    ...
```

Read it like English

- Module practical uses /module-orders + M05_CUSTOMERS.csv.
- Source intentionally has negative amount, unknown customer and duplicate key.
- Evidence must explain what happened even after the run is over.
- A failed required page means run INCOMPLETE/FAILED, not SUCCESS.

Expected check

evidence	must show
extract	page counts + completion
raw	saved pages
validation	valid/rejected + reasons

evidence	must show
load	target/upsert count
rerun	no unintended duplicate keys
log	start/end/status/counts, no secrets

Common beginner traps

- Transforming before preserving any raw evidence.
- Dropping bad rows silently.
- Calling partial pages success.
- Doubling target on rerun.
- Logging credentials.

Now you do a similar task

Complete the module practical only after saving an independent attempt. Explain source, grain, rules, counts, failure state and rerun behavior.

How to prove it

- Save actual code/command.
- Save one independent validation.
- State important input/output grain or item meaning.
- Debug disagreements before moving on.

STOP - check your understanding

Name the M05 pipeline stages and one reconciliation that proves completeness.

Do not turn the page until you can answer in your own words.